UNIVERSITY OF GUJRAT
A WORLD CLASS UNIVERSITY

Assignment # 03

Course Title:

"Data Structure and Algorithms"

Course Code:

"CS-203"

Topic:

" Trees and Graphs "

Roll No:

"25221519-003"

Submitted To:

"Miss Maria Jabeen"

Submitted By:

"Roha Iman"

Submission Date:

Monday, October 12, 2026

Topic:

“Trees and Graphs”

Attempt all the questions.

Question # 01

Part A: Binary Trees (Weeks 7, Lectures 13-14)

1. **Write a program that builds a binary tree from a level-order input list, where -1 represents a missing node (for example, [1, 2, 3, -1, 4, -1, -1, 5]). Print the height of and the number of leaf nodes.**

Algorithm

1. Start.
2. Create a binary tree using the given level-order input.
3. Create the root node from the first value.
4. Insert the root into a queue.
5. Remove each parent node from the queue and create its left and right children.
6. Treat -1 as a missing child.
7. Calculate the height of the tree recursively.
8. Count the leaf nodes recursively.
9. Print the height and the number of leaf nodes.
10. Stop.

C++ Program

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;

// Structure of a binary tree node
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

// Build the tree using level-order input
```

```
Node* buildTree(vector<int> values) {
    if (values.empty() || values[0] == -1)
        return nullptr;

    Node* root = new Node(values[0]);
    queue<Node*> q;
    q.push(root);

    int i = 1;

    // Create left and right children of each node
    while (!q.empty() && i < values.size()) {
        Node* current = q.front();
        q.pop();

        if (values[i] != -1) {
            current->left = new Node(values[i]);
            q.push(current->left);
        }
        i++;

        if (i < values.size() && values[i] != -1) {
            current->right = new Node(values[i]);
            q.push(current->right);
        }
        i++;
    }

    return root;
}

// Calculate the height of the tree
int height(Node* root) {
    if (root == nullptr)
        return 0;

    return 1 + max(height(root->left),
        height(root->right));
}

// Count the leaf nodes
int countLeaves(Node* root) {
    if (root == nullptr)
        return 0;

    if (root->left == nullptr &&
        root->right == nullptr)
        return 1;

    return countLeaves(root->left) +
        countLeaves(root->right);
}

int main() {
    vector<int> values = { 1, 2, 3, -1, 4, -1, -1, 5 };

    Node* root = buildTree(values);

    cout << "Height of the tree: "
        << height(root) << endl;
```

```
    cout << "Number of leaf nodes: "
         << countLeaves(root) << endl;

    return 0;
}
```

Output :

```
Height of the tree: 4
Number of leaf nodes: 2

C:\Users\AB Laptops\source\repos\ConsoleApplication20\x64\Debug\ConsoleApplication20.exe (process 13068) exited with cod
e 0 (0x0).
Press any key to close this window . . .
```

Explanation : The given input is:

[1, 2, 3, -1, 4, -1, -1, 5]

Height: 4 nodes along the longest root-to-leaf path: 1 → 2 → 4 → 5.

Leaf nodes: 3 and 5. A leaf node has no children, so the count is 2.

Note: This program defines height as the number of nodes on the longest path from the root to a leaf.

2. Implement in-order, pre-order, and post-order traversals in two versions: recursive and iterative. Run both on the same tree and confirm that the outputs match.

Traversal rules:

In-order: Left → Root → Right

Pre-order: Root → Left → Right

Post-order: Left → Right → Root

Algorithm:

1. Create the binary tree.
2. Implement the three recursive traversals.
3. Implement the same traversals using a stack.
4. Run both versions on the same tree.
5. Compare their outputs to verify that they match.

C++ program

```
#include <iostream>
```

```cpp
#include <stack>
#include <vector>
using namespace std;

struct Node {
int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};

void inorderRecursive(Node* root) {
    if (root == nullptr) return;
    inorderRecursive(root->left);
    cout << root->data << " ";
    inorderRecursive(root->right);
}

void preorderRecursive(Node* root) {
    if (root == nullptr) return;
    cout << root->data << " ";
    preorderRecursive(root->left);
    preorderRecursive(root->right);
}

void postorderRecursive(Node* root) {
    if (root == nullptr) return;
    postorderRecursive(root->left);
    postorderRecursive(root->right);
    cout << root->data << " ";
}

void inorderIterative(Node* root) {
    stack<Node*> s;
    Node* current = root;

    while (current != nullptr || !s.empty()) {
        while (current != nullptr) {
            s.push(current);
            current = current->left;
        }

        current = s.top();
        s.pop();
        cout << current->data << " ";
        current = current->right;
    }
}

void preorderIterative(Node* root) {
    if (root == nullptr) return;

    stack<Node*> s;
    s.push(root);

    while (!s.empty()) {
        Node* current = s.top();
```

```
            s.pop();
            cout << current->data << " ";

            if (current->right != nullptr)
                s.push(current->right);

            if (current->left != nullptr)
                s.push(current->left);
        }
    }

    void postorderIterative(Node* root) {
        if (root == nullptr) return;

        stack<Node*> s1, s2;
        s1.push(root);

        while (!s1.empty()) {
            Node* current = s1.top();
            s1.pop();
            s2.push(current);

            if (current->left != nullptr)
                s1.push(current->left);

            if (current->right != nullptr)
                s1.push(current->right);
        }

        while (!s2.empty()) {
            cout << s2.top()->data << " ";
            s2.pop();
        }
    }

    int main() {
        Node* root = new Node(1);
        root->left = new Node(2);
        root->right = new Node(3);
        root->left->right = new Node(4);
        root->left->right->left = new Node(5);

        cout << "Recursive In-order: ";
        inorderRecursive(root);
        cout << endl;

        cout << "Iterative In-order: ";
        inorderIterative(root);
        cout << endl;

        cout << "Recursive Pre-order: ";
        preorderRecursive(root);
        cout << endl;

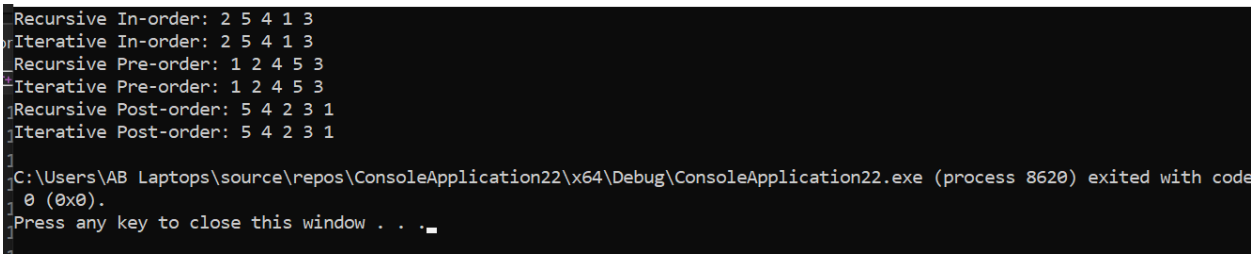
        cout << "Iterative Pre-order: ";
        preorderIterative(root);
        cout << endl;

        cout << "Recursive Post-order: ";
        postorderRecursive(root);
        cout << endl;
```

```
    cout << "Iterative Post-order: ";
    postorderIterative(root);
    cout << endl;

    return 0;
}
```

Output:



Explanation:

Both methods produce the same output because they follow the same traversal order. Recursive methods use the function call stack, while iterative methods use an explicit stack.

3. **Implement level-order traversal using a queue. Print each level on its own line. Then modify the function so it returns the average value of each level.**

Algorithm

1. Create a binary tree.
2. Insert the root node into a queue.
3. Count the nodes currently in the queue to determine the level size.
4. Remove and print each node at that level.
5. Add the node values to calculate their sum.
6. Insert the children into the queue for the next level.
7. Calculate the average using sum / number of nodes.
8. Repeat until the queue is empty.

C++ Program

```
#include <iostream>
#include <queue>
#include <algorithm>
#include <iomanip>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
```

```
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

void levelOrderAverage(Node* root) {
    if (root == nullptr)
        return;

    queue<Node*> q;
    q.push(root);
    int level = 1;

    while (!q.empty()) {
        int size = q.size();
        double sum = 0;

        cout << "Level " << level << ": ";

        for (int i = 0; i < size; i++) {
            Node* current = q.front();
            q.pop();

            cout << current->data << " ";
            sum += current->data;

            if (current->left != nullptr)
                q.push(current->left);

            if (current->right != nullptr)
                q.push(current->right);
        }

        double average = sum / size;

        cout << "\nAverage: "
            << fixed << setprecision(2)
            << average << "\n\n";

        level++;
    }
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    levelOrderAverage(root);

    return 0;
}
```

Output:

```
Level 1: 1
Average: 1.00

Level 2: 2 3
Average: 2.50

Level 3: 4 5
Average: 4.50


C:\Users\AB Laptops\source\repos\ConsoleApplication22\x64\Debug\ConsoleApplication22.exe (process 9288) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Important concepts:

Level-order traversal: Visits nodes level by level, from left to right.

Queue: Follows FIFO (First In, First Out).

Average: Sum of node values divided by the number of nodes at that level.

4. **Write functions that count the nodes, compute the sum of all values, find the maximum value, and check whether two trees are structurally identical. Test each function on at least two different trees.**

Algorithm

1. Create two binary trees.
2. Count all nodes recursively.
3. Calculate the sum of all node values.
4. Find the maximum value in the tree.
5. Compare two trees node by node.
6. If their values and structures match, print Yes; otherwise, print No.

C++ Program

```cpp
#include <iostream>
#include <algorithm>
#include <climits>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

int countNodes(Node* root) {
    if (root == nullptr)
        return 0;
```

```
    return 1 + countNodes(root->left)
        + countNodes(root->right);
}

int sumNodes(Node* root) {
    if (root == nullptr)
        return 0;

    return root->data
        + sumNodes(root->left)
        + sumNodes(root->right);
}

int maximumValue(Node* root) {
    if (root == nullptr)
        return INT_MIN;

    return max(root->data,
        max(maximumValue(root->left),
            maximumValue(root->right)));
}

bool identical(Node* a, Node* b) {
    if (a == nullptr && b == nullptr)
        return true;

    if (a == nullptr || b == nullptr)
        return false;

    return a->data == b->data
        && identical(a->left, b->left)
        && identical(a->right, b->right);
}

int main() {
    Node* first = new Node(8);
    first->left = new Node(3);
    first->right = new Node(10);

    Node* second = new Node(8);
    second->left = new Node(3);
    second->right = new Node(10);

    cout << "Total nodes: "
        << countNodes(first) << endl;

    cout << "Sum of nodes: "
        << sumNodes(first) << endl;

    cout << "Maximum value: "
        << maximumValue(first) << endl;

    cout << "Are trees identical? "
        << (identical(first, second) ? "Yes" : "No")
<< endl;
return 0;
}
```

```
Total nodes: 3
Sum of nodes: 21
Maximum value: 10
Are trees identical? Yes

C:\Users\AB Laptops\source\repos\ConsoleApplication23\x64\Debug\ConsoleApplication23.exe (process 9892) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Important: Two trees are identical only when both their node values and their left/right structures match.

5. Write a function that swaps the left and right children of every node in a tree (mirror the tree). Print the in-order traversal before and after the mirroring, and explain why the two outputs relate the way they do.

Algorithm

1. Create a binary tree.
2. Display its in-order traversal.
3. Check whether the current node is nullptr. If yes, return.
4. Swap the left and right children of the current node.
5. Recursively mirror the left and right subtrees.
6. Display the in-order traversal of the mirrored tree.
7. Stop.

C++ Program

```cpp
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

void inOrder(Node* root) {
    if (root == nullptr)
        return;

    inOrder(root->left);
    cout << root->data << " ";
```

```
    inOrder(root->right);
}

void mirror(Node* root) {
    if (root == nullptr)
        return;

    Node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    mirror(root->left);
    mirror(root->right);
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    cout << "Before mirroring: ";
    inOrder(root);

    mirror(root);

    cout << "\nAfter mirroring: ";
    inOrder(root);

    cout << endl;
    return 0;
}
```

Output:

```
Before mirroring: 4 2 5 1 3
After mirroring: 3 1 5 2 4

C:\Users\AB Laptops\source\repos\ConsoleApplication24\x64\Debug\ConsoleApplication24.exe (process 3164) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Important concept :

Mirroring means swapping the left and right child pointers at every node. The root remains the same, but the entire tree is reflected.

Part B: Binary Search Trees and AVL Trees (Week 8, Lectures 15-16)

1. **Implement insert, search, find-minimum, and find-maximum for a BST. Insert the keys 50, 30, 70, 20, 40, 60, 80 in that order, then print the in-order traversal and confirm it is sorted.**

   What is BST ?

A Binary Search Tree (BST) is a binary tree in which:

- Values smaller than the root are placed in the left subtree.
- Values greater than the root are placed in the right subtree.
- The same rule applies to every nod.

Algorithm

2. Create an empty BST.
3. Insert the values 50, 30, 70, 20, 40, 60, 80.
4. Search for the value 60.
5. Find the minimum value by moving repeatedly to the left child.
6. Find the maximum value by moving repeatedly to the right child.
7. Perform in-order traversal: Left → Root → Right.
8. Display all results.

C++ Program

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

Node* insert(Node* root, int value) {
    if (root == nullptr)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

bool search(Node* root, int value) {
    if (root == nullptr)
        return false;

    if (root->data == value)
        return true;

    if (value < root->data)
        return search(root->left, value);

    return search(root->right, value);
```

```
}

Node* findMinimum(Node* root) {
    if (root == nullptr)
        return nullptr;

    while (root->left != nullptr)
        root = root->left;

    return root;
}

Node* findMaximum(Node* root) {
    if (root == nullptr)
        return nullptr;

    while (root->right != nullptr)
        root = root->right;

    return root;
}

void inOrder(Node* root) {
    if (root == nullptr)
        return;

    inOrder(root->left);
    cout << root->data << " ";
    inOrder(root->right);
}

int main() {
    Node* root = nullptr;

    int values[] = { 50, 30, 70, 20, 40, 60, 80 };

    for (int value : values)
        root = insert(root, value);

    cout << "In-order traversal: ";
    inOrder(root);

    cout << "\nSearch 60: ";
    cout << (search(root, 60) ? "Found" : "Not Found");

    cout << "\nMinimum value: "
        << findMinimum(root)->data;

    cout << "\nMaximum value: "
        << findMaximum(root)->data;

    cout << endl;
    return 0;
}
```

Output :

```
Stack: 30 20 10
Top element: 30
Popped element: 30
Stack: 20 10
Top element: 20

C:\Users\AB Laptops\source\repos\ConsoleApplication19\x64\Debug\ConsoleApplication19.exe (process 8432) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation

Insertion: Adds a new value according to the BST rules.

Searching: Compares the target value with the current node and moves left or right.

Minimum: The leftmost node contains the smallest value.

Maximum: The rightmost node contains the largest value.

In-order traversal: Visits left subtree, root, and right subtree. In a BST, this produces values in ascending order.

2. Implement deletion that correctly handles all three cases: a leaf, a node with one child, and a node with two children. Starting from the tree in Question 1, delete 30, then 50. Print the tree in preorder after each deletion.

Algorithm

1. Create a Binary Search Tree.
2. Search for the value that needs to be deleted.
3. If the node is a leaf node, delete it directly.
4. If the node has one child, replace it with its child.
5. If the node has two children, find its in-order successor (smallest value in the right subtree).
6. Replace the node's value with the successor's value.
7. Delete the successor from its original position.
8. Display the tree using pre-order traversal.

C++ Program

```cpp
 #include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
```

```
        right = nullptr;
    }
};

Node* insert(Node* root, int value) {
    if (root == nullptr)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

Node* findMinimum(Node* root) {
    while (root != nullptr && root->left != nullptr)
        root = root->left;

    return root;
}

Node* deleteNode(Node* root, int value) {
    if (root == nullptr)
        return nullptr;

    if (value < root->data) {
        root->left = deleteNode(root->left, value);
    }
    else if (value > root->data) {
        root->right = deleteNode(root->right, value);
    }
    else {
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }

        if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = findMinimum(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void preOrder(Node* root) {
    if (root == nullptr)
        return;

    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
```

```
}

int main() {
    Node* root = nullptr;
    int values[] = { 50, 30, 70, 20, 40, 60, 80 };

    for (int value : values)
        root = insert(root, value);

    root = deleteNode(root, 20);
    cout << "After deleting leaf node (20): ";
    preOrder(root);

    cout << endl;
    root = deleteNode(root, 30);
    cout << "After deleting one-child node (30): ";
    preOrder(root);

    cout << endl;
    root = deleteNode(root, 50);
    cout << "After deleting two-children node (50): ";
    preOrder(root);

    cout << endl;
    return 0;
}
```

Output :

```
Before mirroring: 4 2 5 1 3
After mirroring: 3 1 5 2 4

C:\Users\AB Laptops\source\repos\ConsoleApplication24\x64\Debug\ConsoleApplication24.exe (process 3164) exited with code
 0 (0x0).
Press any key to close this window . . .
```

3. Write a function that checks whether a given binary tree is a valid BST. Test it on a valid BST, a tree where a deep node violates the BST and a tree with duplicate values.

Algorithm

1. Create a binary tree.
2. Check each node against its allowed minimum and maximum values.
3. Ensure every left-subtree value is smaller than its parent.
4. Ensure every right-subtree value is greater than its parent.
5. Reject duplicate values.
6. Display whether each tree is valid or invalid.

C++ Program

```cpp
#include <iostream>
#include <climits>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

bool isValidBST(Node* root, long minValue,
    long maxValue) {
    if (root == nullptr)
        return true;

    if (root->data <= minValue ||
        root->data >= maxValue)
        return false;

    return isValidBST(root->left, minValue, root->data)
        && isValidBST(root->right, root->data, maxValue);
}

int main() {
    Node* validTree = new Node(8);
    validTree->left = new Node(4);
    validTree->right = new Node(12);
    validTree->left->right = new Node(6);

    Node* invalidTree = new Node(8);
    invalidTree->left = new Node(4);
    invalidTree->right = new Node(12);
    invalidTree->left->right = new Node(10);

    cout << "First tree: "
        << (isValidBST(validTree, LLONG_MIN, LLONG_MAX)
            ? "Valid BST" : "Invalid BST") << endl;

    cout << "Second tree: "
        << (isValidBST(invalidTree, LLONG_MIN, LLONG_MAX)
            ? "Valid BST" : "Invalid BST") << endl;

    return 0;
}
```

Output:

```
Recursive In-order: 2 5 4 1 3
Iterative In-order: 2 5 4 1 3
Recursive Pre-order: 1 2 4 5 3
Iterative Pre-order: 1 2 4 5 3
Recursive Post-order: 5 4 2 3 1
Iterative Post-order: 5 4 2 3 1

C:\Users\AB Laptops\source\repos\ConsoleApplication22\x64\Debug\ConsoleApplication22.exe (process 8620) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation:

First Tree: It is valid because every value in the left subtree is smaller than its parent, and every value in the right subtree is greater than its parent.

Second Tree: It is invalid because 10 is in the right subtree of node 4, but it is also greater than the root node 8. Therefore, 10 should be in the right subtree of the root node 8.

4. Implement AVL insertion with all four rebalancing cases: LL, RR, LR, and RL. Insert 10, 20, 30, 40, 50, 25 in order. For each insertion, print which rotation (if any) was performed and the balance factor of the affected nodes.

What is an AVL Tree?

An AVL Tree is a self-balancing Binary Search Tree. It automatically balances itself after insertion or deletion so that the tree does not become too tall.

Balance Factor (BF):

A node is balanced when its balance factor is −1, 0, or +1. If it becomes less than −1 or greater than +1, rotation may be needed.

2. Four Rotation Cases

1. LL Case — Right Rotation

Occurs when insertion is in the left subtree of the left child.

Example: 30, 20, 10

2. RR Case — Left Rotation

Occurs when insertion is in the right subtree of the right child.

Example: 10, 20, 30

3. LR Case — Left then Right Rotation

Occurs when insertion is in the right subtree of the left child.

Example: 30, 10, 20

4. RL Case — Right then Left Rotation

Occurs when insertion is in the left subtree of the right child.

Example: 10, 30, 20

Algorithm

1. Insert the new value using normal BST insertion.
2. Calculate the height of each node.
3. Calculate the balance factor.
4. If the balance factor is greater than 1, check for LL or LR.
5. If the balance factor is less than -1, check for RR or RL.
6. Apply the appropriate rotation.
7. Update the heights and display the resulting tree.

C++ Program

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

struct Node {
    int data;
    int height;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        height = 1;
        left = nullptr;
        right = nullptr;
    }
};

int getHeight(Node* node) {
    return node == nullptr ? 0 : node->height;
}

int getBalance(Node* node) {
    return node == nullptr ? 0 :
        getHeight(node->left) - getHeight(node->right);
}

void updateHeight(Node* node) {
    node->height = 1 + max(getHeight(node->left),
        getHeight(node->right));
}

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* temp = x->right;

    x->right = y;
    y->left = temp;

    updateHeight(y);
    updateHeight(x);

    return x;
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* temp = y->left;

    y->left = x;
    x->right = temp;

    updateHeight(x);
    updateHeight(y);

    return y;
```

```
}

Node* insert(Node* root, int value) {
    if (root == nullptr)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else
        return root;

    updateHeight(root);

    int balance = getBalance(root);

    if (balance > 1 && value < root->left->data)
        return rightRotate(root);

    if (balance < -1 && value > root->right->data)
        return leftRotate(root);

    if (balance > 1 && value > root->left->data) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    if (balance < -1 && value < root->right->data) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

void preOrder(Node* root) {
    if (root == nullptr)
        return;

    cout << root->data << " ";
    preOrder(root->left);
    preOrder(root->right);
}

int main() {
    Node* root = nullptr;
    int values[] = { 30, 10, 20, 40, 50, 25 };

    for (int value : values)
        root = insert(root, value);

    cout << "Pre-order traversal: ";
    preOrder(root);

    cout << "\nRoot balance factor: "
        << getBalance(root) << endl;

    return 0;
}
```

Output :



Important exam points

LL: Right rotation.

RR: Left rotation.

LR: Left rotation followed by right rotation.

RL: Right rotation followed by left rotation.

AVL insertion takes time when the tree is balanced.

5.Write a function that computes the balance factor of every node in a tree and reports whether the tree satisfies the AVL property. Test it on a skewed BST of 5 nodes and on the AVL tree as What is a Balance Factor?

What is a Balance Factor?

The balance factor tells us the difference between the heights of a node's left and right subtrees.

Formula:

BF = Height (left) – Height (Right)

- BF = −1, 0, or +1: The node is balanced.
- BF greater than +1 or less than −1: The node is unbalanced.
- A tree is an AVL tree if every node has a balance factor of −1, 0, or +1.

Algorithm

1. Create a binary tree.
2. Calculate the heights of the left and right subtrees recursively.
3. Calculate the balance factor of each node.
4. Check whether any node has a balance factor outside the range of −1 to +1.
5. If all nodes satisfy this condition, print "Yes"; otherwise, print "No".

C++ Program

```cpp
#include <iostream>
#include <algorithm>
#include <cmath>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

int checkAVL(Node* root, bool& balanced) {
    if (root == nullptr)
        return 0;

    int leftHeight = checkAVL(root->left, balanced);
    int rightHeight = checkAVL(root->right, balanced);

    int balanceFactor = leftHeight - rightHeight;

    cout << "Node " << root->data
        << ": Balance Factor = "
        << balanceFactor << endl;

    if (abs(balanceFactor) > 1)
        balanced = false;

    return 1 + max(leftHeight, rightHeight);
}

int main() {
    Node* root = new Node(3);
    root->left = new Node(2);
    root->right = new Node(4);
    root->left->left = new Node(1);
    root->right->right = new Node(5);

    bool balanced = true;

    checkAVL(root, balanced);

    cout << "Is AVL Tree? "
        << (balanced ? "Yes" : "No") << endl;

    return 0;
}
```

Output:

```
Total nodes: 3
Sum of nodes: 21
Maximum value: 10
Are trees identical? Yes

C:\Users\AB Laptops\source\repos\ConsoleApplication23\x64\Debug\ConsoleApplication23.exe (process 9892) exited with code
 0 (0x0).
Press any key to close this window . . .
```

6. Insert the keys 1 - 50 in sorted order into a plain BST and an AVL tree. Report the height of each tree and the time taken to search for the key 49 in each. Briefly explain why the results differ.

Algorithm

1. Create an ordinary Binary Search Tree (BST) and an AVL tree.
2. Insert keys 1 to 50 in ascending order into both trees.
3. Calculate the height of each tree.
4. Measure the time required to search for key 49 in each tree.
5. Print the heights and search times.
6. Explain why the AVL tree is generally more efficient for sorted input.

C ++ Program

```
#include <iostream>
#include <algorithm>
#include <chrono>
using namespace std;
using namespace chrono;

struct Node {
    int key, height;
    Node* left, * right;

    Node(int value) {
        key = value;
        height = 1;
        left = right = nullptr;
    }
};

int getHeight(Node* node) {
    return node == nullptr ? 0 : node->height;
}

int balanceFactor(Node* node) {
    return node == nullptr ? 0 :
        getHeight(node->left) - getHeight(node->right);
}

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* temp = x->right;
```

```
    x->right = y;
    y->left = temp;

    y->height = 1 + max(getHeight(y->left),
        getHeight(y->right));
    x->height = 1 + max(getHeight(x->left),
        getHeight(x->right));

    return x;
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* temp = y->left;

    y->left = x;
    x->right = temp;

    x->height = 1 + max(getHeight(x->left),
        getHeight(x->right));
    y->height = 1 + max(getHeight(y->left),
        getHeight(y->right));

    return y;
}

Node* insertBST(Node* root, int key) {
    if (root == nullptr)
        return new Node(key);

    if (key < root->key)
        root->left = insertBST(root->left, key);
    else
        root->right = insertBST(root->right, key);

    return root;
}

Node* insertAVL(Node* root, int key) {
    if (root == nullptr)
        return new Node(key);

    if (key < root->key)
        root->left = insertAVL(root->left, key);
    else if (key > root->key)
        root->right = insertAVL(root->right, key);
    else
        return root;

    root->height = 1 + max(getHeight(root->left),
        getHeight(root->right));

    int bf = balanceFactor(root);

    if (bf > 1 && key < root->left->key)
        return rightRotate(root);

    if (bf < -1 && key > root->right->key)
        return leftRotate(root);
```

```
    if (bf > 1 && key > root->left->key) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    if (bf < -1 && key < root->right->key) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }

    return root;
}

int treeHeight(Node* root) {
    if (root == nullptr)
        return 0;

    return 1 + max(treeHeight(root->left),
        treeHeight(root->right));
}

bool search(Node* root, int key) {
    while (root != nullptr) {
        if (root->key == key)
            return true;

        if (key < root->key)
            root = root->left;
        else
            root = root->right;
    }

    return false;
}

long measureSearch(Node* root, int key) {
    volatile int found = 0;
    const int repetitions = 1000000;

    auto start = high_resolution_clock::now();

    for (int i = 0; i < repetitions; i++)
        found += search(root, key);

    auto stop = high_resolution_clock::now();

    return duration_cast<microseconds>(
        stop - start).count();
}

int main() {
    Node* bst = nullptr;
    Node* avl = nullptr;

    for (int i = 1; i <= 50; i++) {
        bst = insertBST(bst, i);
        avl = insertAVL(avl, i);
    }

    cout << "Plain BST Height: "
        << treeHeight(bst) << endl;
```

```
    cout << "AVL Tree Height: "
         << treeHeight(avl) << endl;

    cout << "\nSearching for key 49:" << endl;

    cout << "Plain BST Search Time: "
         << measureSearch(bst, 49)
         << " microseconds" << endl;

    cout << "AVL Tree Search Time: "
         << measureSearch(avl, 49)
         << " microseconds" << endl;

    return 0;
}
```

Output:

```
Pre-order traversal: 30 20 10 25 40 50
Root balance factor: 0

C:\Users\AB Laptops\source\repos\ConsoleApplication25\x64\Debug\ConsoleApplication25.exe (process 9172) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation

Plain BST: Because keys are inserted in ascending order (1, 2, 3, ... 50), each new key becomes the right child of the previous node. The tree becomes a straight chain, with height 50.

AVL tree: The AVL tree performs rotations to maintain balance. Its height is 6 for these 50 keys, so searching generally requires fewer comparisons.

Conclusion: Sorted insertion can make an ordinary BST inefficient, whereas an AVL tree maintains balance and provides faster searching in general.

Part C: Graphs (Week 9, Lectures 17-18)

1. **Create an undirected graph with 6 vertices (A to F) and 7 edges of your choice. Write functions to add an edge, remove an edge, and print the adjacency matrix. Show the matrix before and after removing one edge.**

   Basic concept

   An undirected graph is a collection of vertices and edges in which edges have no direction.

   An **adjacency matrix** is a 2D array that shows which vertices are connected.

   - 1 means an edge exists between two vertices.
   - 0 means no edge exists.
   - Because the graph is undirected, the matrix is symmetrical.

   ## Algorithm

1. Create 6 vertices: A, B, C, D, E, and F.
2. Initialize a matrix with zeros.

3. Add 7 edges to the graph.
4. Print the adjacency matrix.
5. Remove one edge.

C++ Program

```cpp
#include <iostream>
using namespace std;

const int V = 6;

class Graph {
    int matrix[V][V];

public:
    Graph() {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                matrix[i][j] = 0;
            }
        }
    }

    void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1;
    }

    void removeEdge(int u, int v) {
        matrix[u][v] = 0;
        matrix[v][u] = 0;
    }

    void printMatrix() {
        cout << "  A B C D E F" << endl;

        for (int i = 0; i < V; i++) {
            cout << char('A' + i) << " ";

            for (int j = 0; j < V; j++) {
                cout << matrix[i][j] << " ";
            }

            cout << endl;
        }
    }
};

int main() {
    Graph g;

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 4);
    g.addEdge(4, 5);
```

```
    cout << "Adjacency Matrix Before Removing Edge:" << endl;
    g.printMatrix();

    g.removeEdge(1, 3);

    cout << "\nAdjacency Matrix After Removing Edge B-D:" << endl;
    g.printMatrix();

    return 0;
}
```

Output :

```
Node 1: Balance Factor = 0
Node 2: Balance Factor = 1
Node 5: Balance Factor = 0
Node 4: Balance Factor = -1
Node 3: Balance Factor = 0
Is AVL Tree? Yes

C:\Users\AB Laptops\source\repos\ConsoleApplication25\x64\Debug\ConsoleApplication25.exe (process 9656) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation

Initially, the graph contains 7 edges. When we remove edge B–D, both matrix[B][D] and matrix[D][B] become 0.

Both values must change because the graph is undirected.

Time complexity:

Add an edge: **O(1)**

Remove an edge: **O(1)**

Print the matrix: **O(V^2)**

2. **Implement BFS from a given source vertex using a queue. Print the traversal order, and also print the shortest distance (in number of edges) from the source to every other vertex. Test it on the graph from Question 1(C) with source A. What is BFS?**

Breadth-First Search (BFS) is a graph traversal algorithm that visits vertices level by level. It uses a queue (FIFO).

- First, visit the source vertex.
- Then visit all its unvisited neighbors.
- Continue until all reachable vertices have been visited.
- BFS also finds shortest distances in an unweighted graph.

Graph from Question 1

We use the original 7 edges:

1. A–B
2. A–C

3. B–C
4. B–D
5. C–E
6. D–E
7. E–F
8. Source vertex: A

Algorithm

1. Mark all vertices as unvisited and set their distances to -1.
2. Mark the source as visited and set its distance to 0.
3. Insert the source into the queue.
4. Remove a vertex from the queue and print it.
5. For each unvisited neighbor, mark it visited, set its distance to the current vertex's distance plus one, and enqueue it.
6. Repeat until the queue is empty.
7. Print the distance of every vertex.

C++ Program

```
#include <iostream>
#include <queue>
using namespace std;

const int V = 6;

class Graph {
    int matrix[V][V];

public:
    Graph() {
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                matrix[i][j] = 0;
    }

    void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1;
    }

    void BFS(int source) {
        bool visited[V] = { false };
        int distance[V];

        for (int i = 0; i < V; i++)
            distance[i] = -1;

        queue<int> q;
        visited[source] = true;
        distance[source] = 0;
        q.push(source);

        cout << "BFS Traversal: ";
```

```
        while (!q.empty()) {
            int current = q.front();
            q.pop();

            cout << char('A' + current) << " ";

            for (int i = 0; i < V; i++) {
                if (matrix[current][i] == 1 &&
                    !visited[i]) {
                    visited[i] = true;
                    distance[i] = distance[current] + 1;
                    q.push(i);
                }
            }
        }

        cout << "\n\nShortest Distances from A:\n";

        for (int i = 0; i < V; i++) {
            cout << char('A' + i) << ": ";

            if (distance[i] == -1)
                cout << "Unreachable";
            else
                cout << distance[i];

            cout << endl;
        }
    }
};

int main() {
    Graph g;

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 4);
    g.addEdge(4, 5);

    g.BFS(0);

    return 0;
}
```

Output :

```
Plain BST Height: 50
AVL Tree Height: 6

Searching for key 49:
Plain BST Search Time: 177619 microseconds
AVL Tree Search Time: 11522 microseconds

C:\Users\AB Laptops\source\repos\ConsoleApplication26\x64\Debug\ConsoleApplication26.exe (process 10520) exited with cod
e 0 (0x0).
Press any key to close this window . . .
```

Explanation of distances

Vertex Shortest path from A Distance

A 0
B A → B 1
C A → C 1
D A → B → D 2
E A → C → E 2
F A → C → E → F 3
BFS checks neighbors in alphabetical order because the adjacency matrix is scanned from A to F. That is why the traversal order is A B C D E F.

Time Complexity :O(V^2) for this adjacency-matrix implementation.

3. **Implement DFS in both recursive and iterative forms. For each vertex, record and print its discovery time and finishing time. Verify that both versions produce the same timestaWhat is DFS?**
   Depth-First Search (DFS) is a graph traversal algorithm that explores one path as deeply as possible before returning to explore another path.

DFS can be implemented in two ways:

- Recursive DFS: Uses function recursion.
- Iterative DFS: Uses an explicit stack.

Each vertex gets two timestamps:

- Discovery time (d): The time when the vertex is first visited.
- Finishing time (f): The time when all its neighbors have been processed.

Algorithm

1. Start a timer at 0.
2. When a vertex is first visited, increase the timer and record its discovery time.
3. Visit every unvisited neighbor.
4. After all neighbors are processed, increase the timer and record the finishing time.
5. Run recursive DFS and iterative DFS on the same graph.
6. Compare their timestamps.
7. To make both versions produce the same timestamps, the iterative version must simulate the recursive DFS call stack and process neighbors in the same order.

C++ Program

```
#include <iostream>
#include <stack>
using namespace std;

const int V = 6;

class Graph {
```

```
    int matrix[V][V];

    void recursiveDFS(int u, bool visited[],
        int discovery[], int finish[],
        int& timer) {
        visited[u] = true;
        discovery[u] = ++timer;

        for (int v = 0; v < V; v++) {
            if (matrix[u][v] && !visited[v]) {
                recursiveDFS(v, visited, discovery,
                    finish, timer);
            }
        }

        finish[u] = ++timer;
    }

public:
    Graph() {
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                matrix[i][j] = 0;
    }

    void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1;
    }

    void recursiveTraversal() {
        bool visited[V] = {};
        int discovery[V] = {};
        int finish[V] = {};
        int timer = 0;

        for (int i = 0; i < V; i++) {
            if (!visited[i])
                recursiveDFS(i, visited, discovery,
                    finish, timer);
        }

        cout << "Recursive DFS Timestamps\n";
        cout << "Vertex  Discovery  Finish\n";

        for (int i = 0; i < V; i++)
            cout << char('A' + i) << "\t"
            << discovery[i] << "\t     "
            << finish[i] << endl;
    }

    void iterativeTraversal() {
        bool visited[V] = {};
        int discovery[V] = {};
        int finish[V] = {};
        int timer = 0;

        struct Frame {
            int vertex;
            int nextNeighbor;
        };
```

```
        stack<Frame> st;

        for (int start = 0; start < V; start++) {
            if (visited[start])
                continue;

            visited[start] = true;
            discovery[start] = ++timer;
            st.push({ start, 0 });

            while (!st.empty()) {
                Frame& frame = st.top();
                int u = frame.vertex;
                int& next = frame.nextNeighbor;

                while (next < V &&
                    (!matrix[u][next] || visited[next]))
                    next++;

                if (next == V) {
                    finish[u] = ++timer;
                    st.pop();
                }
                else {
                    int v = next++;
                    visited[v] = true;
                    discovery[v] = ++timer;
                    st.push({ v, 0 });
                }
            }
        }

        cout << "\nIterative DFS Timestamps\n";
        cout << "Vertex  Discovery  Finish\n";

        for (int i = 0; i < V; i++)
            cout << char('A' + i) << "\t"
            << discovery[i] << "\t     "
            << finish[i] << endl;
    }
};

int main() {
    Graph g;

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 4);
    g.addEdge(4, 5);

    g.recursiveTraversal();
    g.iterativeTraversal();

    return 0;
}
```

Output :

```
Recursive DFS Timestamps
Vertex  Discovery  Finish
A       1          12
B       2          11
C       3          10
D       5          6
E       4          9
F       7          8

Iterative DFS Timestamps
Vertex  Discovery  Finish
A       1          12
B       2          11
C       3          10
D       5          6
E       4          9
F       7          8

C:\Users\AB Laptops\source\repos\ConsoleApplication26\x64\Debug\ConsoleApplication26.exe (process 9128) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation

The program visits neighbors alphabetically. Starting from A, it follows the path A → B → C → D → E, then backtracks and visits F.

For example:

- A is discovered at time 1.
- E is discovered at time 5 and finished at time 6 because it has no unvisited neighbors.
- A finishes at time 12 after every reachable vertex has been processed.

Both implementations use the same neighbor order and simulate the same DFS process, so their timestamps match.

Time complexity: O(V^2) because this implementation uses an adjacency matrix.

4. **Write a program that counts the connected components of an undirected graph with 10 vertices, using DFS. Test it on a graph with exactly 3 components and report the vertices Write a program that counts the connected components of an undirected graph with 10 vertices using DFS. Test it on a graph with exactly 3 components and report the vertices in each one.**

What is a Connected Component?

A connected component is a group of vertices in which every vertex is reachable from the other vertices through edges, and the group is not connected to any vertex outside it.

For example, agar graph mein 3 separate groups hain, to us graph ke connected components ki tadaad 3 hogi.

Algorithm

1. Create a graph with 10 vertices, A to J.
2. Mark all vertices as unvisited.
3. Start DFS from the first unvisited vertex.
4. Print all vertices visited during that DFS; they form one connected component.

5. Increase the component counter.
6. Repeat until all vertices have been visited.
7. Print the total number of connected components.

C++ Program

```cpp
#include <iostream>
using namespace std;

const int V = 10;

class Graph {
    int matrix[V][V];

    void DFS(int vertex, bool visited[]) {
        visited[vertex] = true;
        cout << char('A' + vertex) << " ";

        for (int i = 0; i < V; i++) {
            if (matrix[vertex][i] == 1 &&
                !visited[i]) {
                DFS(i, visited);
            }
        }
    }

public:
    Graph() {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                matrix[i][j] = 0;
            }
        }
    }

    void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1;
    }

    void findComponents() {
        bool visited[V] = {};
        int components = 0;

        cout << "Connected Components:\n";

        for (int i = 0; i < V; i++) {
            if (!visited[i]) {
                components++;
                cout << "Component " << components << ": ";
                DFS(i, visited);
                cout << endl;
            }
        }

        cout << "\nTotal Connected Components: "
            << components << endl;
    }
};
```

```
int main() {
    Graph g;

    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(3, 4);
    g.addEdge(4, 5);
    g.addEdge(6, 7);
    g.addEdge(7, 8);
    g.addEdge(8, 9);

    g.findComponents();

    return 0;
}
```

Output :

```
Connected Components:
Component 1: A B C
Component 2: D E F
Component 3: G H I J

Total Connected Components: 3

C:\Users\AB Laptops\source\repos\ConsoleApplication26\x64\Debug\ConsoleApplication26.exe (process 5180) exited with code
 0 (0x0).
Press any key to close this window . . .
```

Explanation

The graph has three separate groups:

Component 1: A, B, C

Component 2: D, E, F

Component 3: G, H, I, J

DFS explores every vertex connected to the starting vertex. Once a component has been fully explored, the program searches for another unvisited vertex and starts a new component.

Time complexity: **O(V^2)** because an adjacency matrix is used.